Core Java: The Language, Again

Syntax, types, control flow, arrays, strings, I/O

10 topics · 67 flashcards · 37 quiz questions · 35 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Day 1: Core Java: The Language, Again

Syntax, types, control flow, arrays, strings, I/O

10 topics · 67 flashcards · 37 quiz questions · ~4.8 hours

> **Goal.** Shake off the rust. By tonight you can write and dry-run any small Java program on paper — which is exactly what a whiteboard round asks for.

Schedule

- Slot 1 (2h): How Java runs + program structure + data types
- Slot 2 (2h): Operators + control flow + switch/loops
- Slot 3 (2h): Arrays + Strings + StringBuilder + String pool
- Slot 4 (1.5h): I/O + stack/heap + pass-by-value
- Slot 5 (1.5h): Flashcards → Quiz → write 5 programs by hand

1. How Java Actually Runs (JDK / JRE / JVM) Must-know 20 min

This is question #1 in a scary number of fresher interviews, and it's free marks if you can draw it. You write `.java` source. `javac` (the compiler, part of the JDK) turns it into **bytecode** in a `.class` file — bytecode is not machine code, it's an instruction set the JVM understands. The JVM then loads, verifies and executes that bytecode. On most JVMs, hot code is compiled to native machine code at runtime by the JIT compiler.

Key points

- **JVM** (Java Virtual Machine) — the engine that *runs* bytecode. Platform-specific, which is why there's a different JVM download for Windows/Linux/Mac.
- **JRE** (Java Runtime Environment) = JVM + the standard libraries. Enough to *run* Java, not to *write* it.
- **JDK** (Java Development Kit) = JRE + compiler (`javac`), debugger, `javadoc`, `jar`. This is what you install as a developer.
- JDK 11+ ships **without a separate JRE**. If someone asks 'JRE or JDK?' the answer is: JDK, it contains everything.
- **WORA** — Write Once, Run Anywhere: the same `.class` file runs on any OS that has a JVM. The portability belongs to the *bytecode*, not the source.
- Java is **compiled AND interpreted**: compiled to bytecode ahead of time, then interpreted/JIT-compiled by the JVM.

Reference

Term	What it is	Interview one-liner
JVM	Abstract machine that executes bytecode	"It's the runtime engine — it makes Java platform-independent."
JRE	JVM + libraries	"Runtime only — enough to run a Java app."
JDK	JRE + dev tools (javac, jar, javadoc)	"What a developer installs."
Bytecode	Intermediate code in .class files	"Not machine code — the JVM's instruction set."
JIT	Just-In-Time compiler	"Compiles hot bytecode to native code for speed."

Code

Compile and run — the two commands they may ask you to write

```
javac Hello.java     # source  -> bytecode (Hello.class)
java  Hello          # bytecode -> run (NO .class extension!)
java --version       # check which JDK you're on
```

```
Output:
Hello, World!
```

The smallest Java program, annotated

```
// File name MUST match the public class name: Hello.java
public class Hello {
   public static void main(String[] args) {   // the entry point
       System.out.println("Hello, World!");   // println = print + newline
   }
}
```

```
Output:
Hello, World!
```

Why main is public static void main(String[] args)

```
public  -> JVM (outside your package) must be able to call it
static  -> no object exists yet, so it must be callable on the class itself
void    -> returns nothing to the JVM; use System.exit(0) for a status code
String[] args -> command-line arguments (`java MyApp a b` -> args = ["a","b"])

// All of these are legal:
static public void main(String[] args)      // modifiers can be reordered
public static void main(String args[])      // C-style array syntax
public static void main(String... args)     // varargs
// This one is NOT an entry point (no String[]):
public static void main()                   // just an overloaded method
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java is compiled to bytecode by javac, and the JVM interprets or JIT-compiles that bytecode at runtime. That bytecode layer is what gives Java platform independence — the same class file runs anywhere a JVM exists."

Flashcards

Q. Difference between JDK, JRE and JVM?
A. JVM runs bytecode. JRE = JVM + libraries, enough to run. JDK = JRE + dev tools like javac, used to develop.

Q. Is Java compiled or interpreted?
A. Both. javac compiles source to bytecode; the JVM interprets/JIT-compiles that bytecode into native machine code.

Q. What does WORA mean and what makes it possible?
A. Write Once Run Anywhere. The .class bytecode is platform-neutral; each OS has its own JVM that executes it.

Q. Why must main be static?
A. The JVM calls main before any object of the class exists, so it must be invocable on the class itself, not an instance.

Q. Can you overload main()?
A. Yes — you can write many main methods with different parameters, but only public static void main(String[]) is the entry point.

Q. Since Java 11, do you need to install the JRE separately?
A. No. The JDK includes everything; there's no separate JRE download for modern versions.

Quiz

Q1. What is the output of compiling a file named `Test.java` that contains `class A {}` and `public class B {}`?

[ ] A. Compiles fine, produces A.class and B.class
[x] B. Compile error: class B is public, should be declared in a file named B.java
[ ] C. Compiles, produces only B.class
[ ] D. Runtime error

Why: A source file can contain at most one public top-level class, and its name must match the file name. `public class B` in `Test.java` is a compile-time error.

Q2. Which command runs a compiled Java class named `App`?

[ ] A. java App.class
[x] B. java App
[ ] C. javac App
[ ] D. run App

Why: `java` takes the *class name*, not the file name. `java App.class` looks for a class literally called `App.class` and fails with ClassNotFoundException / NoClassDefFoundError.

Q3. `public static void main(String[] args)` — which part can you legally change and still have a valid entry point?

[ ] A. Change void to int
[ ] B. Remove static
[ ] C. Remove public
[x] D. Write `String... args` instead of `String[] args`

> **Why:** Varargs `String... args` compiles to `String[]`, so the signature still matches. Return type, static and public are all part of what the JVM looks for.

Q4. What does the JIT compiler do?

[ ] A. Compiles .java to .class
[x] B. Compiles frequently-executed bytecode into native machine code at runtime
[ ] C. Verifies bytecode is safe
[ ] D. Converts Java to JavaScript

> **Why:** javac does source→bytecode. The JIT sits inside the JVM and compiles hot methods to native code for speed, which is why long-running Java apps get faster after warm-up.

2. Data Types, Variables & Wrappers Must-know 35 min

Java has **8 primitive types** — they are not objects and live on the stack when local. Everything else is a reference to an object on the heap. Knowing the sizes and ranges is a classic rapid-fire question, and the wrapper classes are where the sneaky interview traps live.

Key points

- `byte(1) short(2) int(4) long(8) float(4) double(8) char(2) boolean(~1 bit, JVM-dependent)` — sizes in bytes, and they are the SAME on every platform (that's part of Java's portability promise).
- Whole-number literals are `int` by default; you need the `L` suffix for `long`: `long x = 10000000000L;` won't compile without it.
- Decimal literals are `double` by default; `float f = 3.14;` is a compile error — write `3.14f`.
- `char` is a **16-bit unsigned Unicode** value (0 to 65535), so `char c = 65;` is `'A'` and char arithmetic works.
- Instance/class fields get default values (`0`, `0.0`, `false`, `'\u0000'`, `null`). **Local variables do not** — using one before assignment is a compile error.
- `final` makes a variable a constant (reference can't be reassigned — the object it points to can still mutate).

Reference

Primitive	Size	Wrapper	Range / notes	Default
byte	1 byte	Byte	-128 to 127	0
short	2 bytes	Short	-32,768 to 32,767	0
int	4 bytes	Integer	-2^31 to 2^31-1 (~2.1 billion)	0
long	8 bytes	Long	-2^63 to 2^63-1 — use L suffix	0L
float	4 bytes	Float	~7 decimal digits — use f suffix	0.0f
double	8 bytes	Double	~15 decimal digits, default for decimals	0.0
char	2 bytes	Character	0 to 65,535 (Unicode, unsigned)	'\u0000'
boolean	JVM-dependent	Boolean	true / false — NOT 0/1 in Java	false

Code

Autoboxing, unboxing and the Integer cache trap (VERY common question)

```
Integer a = 127, b = 127;
System.out.println(a == b);      // true  -> same cached object

Integer c = 128, d = 128;
System.out.println(c == d);      // false -> different objects (outside cache)

System.out.println(c.equals(d)); // true  -> compares VALUE (always use this!)

Integer e = new Integer(127), f = new Integer(127);
System.out.println(e == f);      // false -> `new` ALWAYS makes a new object

int g = 128, h = 128;
System.out.println(g == h);      // true  -> primitives compare by value
```

```
Output:
true
false
true
false
true
```

> Integer caches -128..127 (Byte, Short, Long, Character too; Character is 0..127). `new` bypasses the cache. **Never compare wrappers with ==.**

Widening vs narrowing: implicit vs explicit casts

```
int i = 100;
long l = i;          // widening   - automatic, no data loss
double d = l;        // widening   - automatic
// int x = l;        // ERROR - narrowing needs a cast
int x = (int) l;     // explicit cast

double pi = 3.99;
int trunc = (int) pi;          // 3  -> truncates, does NOT round
long rounded = Math.round(3.5);// 4  -> Math.round returns long for double

byte b1 = 10;                  // OK: constant fits in byte
// byte b2 = 200;              // ERROR: 200 > 127
byte b3 = (byte) 200;          // -56 -> overflow wraps around

System.out.println(10 / 3);    // 3   -> integer division!
System.out.println(10.0 / 3);  // 3.3333333333333335
System.out.println(10 % 3);    // 1   -> remainder, works on doubles too
```

```
Output:
3
3.3333333333333335
1
```

Type promotion rules in expressions

```
byte a = 10, b = 20;
// byte c = a + b;      // ERROR! a+b is promoted to int
byte c = (byte) (a + b); // must cast back

System.out.println('A' + 0);        // 65   -> char promotes to int
System.out.println("5" + 5);        // "55" -> int becomes String
System.out.println(5 + 5 + "Java"); // "10Java" -> + is left-to-right
System.out.println("Java" + 5 + 5); // "Java55"
System.out.println(1.0 / 0);        // Infinity (no exception for doubles!)
// System.out.println(1 / 0);       // ArithmeticException: / by zero
```

```
Output:
65
55
10Java
Java55
Infinity
```

> Any arithmetic on byte/short/char promotes to int. String + anything = String concatenation.

Parsing and converting — the methods you'll use in every coding question

```
int n    = Integer.parseInt("42");
double v = Double.parseDouble("3.14");
long big = Long.parseLong("9999999999");
// Integer.parseInt("abc");  -> NumberFormatException (unchecked)

String s1 = String.valueOf(42);   // "42"  (works with null -> "null")
String s2 = Integer.toString(42); // "42"
String s3 = "" + 42;              // "42"  (works but ugly/slow)

Integer boxed   = Integer.valueOf(42);  // uses the cache
int    unboxed  = boxed;                // auto-unboxing
// Integer nullInt = null; int y = nullInt;  -> NullPointerException on unbox!
```

SAY IT LIKE THIS IN THE INTERVIEW

"There are 8 primitives with fixed sizes across platforms; everything else is a reference. Wrappers give me objects for collections and generics, and autoboxing converts automatically — but I always compare them with equals(), because == compares references and the Integer cache only covers -128 to 127."

Flashcards

Q. Why does `Integer a=127, b=127; a==b` print true but `128` prints false?
A. The Integer cache reuses objects for -128..127. Above that range, each boxing creates a new object, so == compares different references.

Q. What's the default value of a local variable vs an instance variable?
A. Instance/class fields get defaults (0, false, null). Local variables have NO default — using one before assigning is a compile error.

Q. Is `boolean` 1 bit in Java?
A. Size is JVM-dependent (often 1 byte in arrays, 4 bytes as a local). What's guaranteed: it holds only true/false, and unlike C, 0/1 are not booleans.

Q. Why does `byte c = a + b;` fail to compile when a and b are bytes?
A. byte/short/char are promoted to int during arithmetic, so the result is int and needs an explicit cast back to byte.

Q. What's the difference between `parseInt` and `valueOf`?
A. parseInt returns a primitive int; valueOf returns an Integer object (and uses the -128..127 cache).

Q. What exception does `Integer.parseInt("abc")` throw?
A. NumberFormatException — it's unchecked (extends IllegalArgumentException / RuntimeException).

Q. What is autoboxing? Give a risk it creates.
A. Automatic primitive↔wrapper conversion (int → Integer). Risk: unboxing a null wrapper throws NullPointerException, and == on wrappers compares references.

Q. final on a variable — what exactly is final?
A. The reference can't be reassigned. The object's internal state can still change (final List can still have elements added).

Quiz

Q1. What does this print? `System.out.println("Result: " + 2 + 3 * 4);`

[ ] A. Result: 14
[x] B. Result: 212
[ ] C. Result: 24
[ ] D. Compile error

Why: `*` binds tighter than `+`, so `3*4 = 12`. Then `"Result: " + 2` → `"Result: 2"`, then `+ 12` → `"Result: 212"`. Evaluation is left-to-right once precedence is applied.

Q2. Which statement about `char` is true?

[ ] A. char is signed and 8-bit
[x] B. char is an unsigned 16-bit Unicode value
[ ] C. char cannot be used in arithmetic

[ ] D. char defaults to '\0' only in arrays

> **Why:** char is 16-bit unsigned (0–65535) UTF-16 code unit. It's the only unsigned primitive in Java, and it participates in arithmetic via int promotion.

Q3. What happens here? `Integer x = null; int y = x;`

[ ] A. y becomes 0
[x] B. NullPointerException at runtime
[ ] C. Compile error
[ ] D. y becomes null

> **Why:** Unboxing calls x.intValue() on a null reference → NPE. This bites in real code when a Map<Integer,Integer> misses a key.

Q4. `long big = 9_000_000_000;` — does this compile?

[ ] A. Yes, underscores are allowed
[x] B. No, needs the L suffix: 9_000_000_000L
[ ] C. No, underscores aren't allowed in Java
[ ] D. No, the value is too large for long

> **Why:** Underscores in numeric literals are legal since Java 7 (readability only). But the literal is still an int by default and 9 billion overflows int, so you need the L suffix.

Q5. What is printed? `double d = 1.0 / 0; System.out.println(d);`

[ ] A. ArithmeticException
[x] B. Infinity
[ ] C. NaN
[ ] D. 0.0

> **Why:** Floating-point division by zero yields Infinity (or NaN for 0.0/0.0), not an exception. Only *integer* division by zero throws ArithmeticException.

3. Operators & Precedence Core 25 min

You use these every day, so the only reason they show up in interviews is the gotchas: `&&` vs `&`, short-circuiting, integer division, and the increment operators.

Key points

- **Short-circuit**: `&&` and `||` stop as soon as the result is known. `&` and `|` on booleans always evaluate both sides.
- Use short-circuiting deliberately — it's the idiomatic null guard: `if (s != null && s.length() > 3)`.
- `&`, `|`, `^` are also **bitwise** operators on integers (`^` swaps two numbers without a temp).
- `>>` keeps the sign, `>>>` fills with zeros (unsigned shift), `<<` multiplies by 2^n.
- `++i` (pre) increments then uses the value; `i++` (post) uses the value then increments.
- Ternary `?:` is right-associative and must be a complete expression, not a statement.

Reference

Operator	Meaning	Gotcha
&& / ||	Short-circuit logical	Skips the right side when possible — the null-guard trick
& / |	Non-short-circuit logical; also bitwise	On booleans it evaluates BOTH sides — can NPE where && wouldn't
^	XOR	Bitwise on ints: a^b^b == a (swap trick)
>> / >>>	Signed / unsigned right shift	>>> fills with 0, so negative numbers become huge positives
?:	Ternary	Right-associative; both branches must have compatible types
instanceof	Type check	Java 16+: `if (o instanceof String s)` binds s directly

Code

Post vs pre increment — the classic dry-run question

```
int i = 5;
System.out.println(i++);   // 5  -> prints, THEN increments
System.out.println(i);     // 6
System.out.println(++i);   // 7  -> increments, THEN prints

int a = 10;
a = a++;                   // a stays 10! (old value assigned back)
System.out.println(a);

int x = 5, y = 5;
System.out.println(x++ + ++x);   // 5 + 7 = 12
System.out.println(y-- - --y);   // 5 - 3 = 2
```

```
Output:
5
6
7
10
12
2
```

Short-circuit saves you — and its absence breaks you

```
String s = null;

if (s != null && s.length() > 3) { }   // safe: right side never runs
// if (s != null & s.length() > 3) { } // NullPointerException!

int n = 10;
if (n > 5 || n / 0 == 1) {            // safe: || stops at true
   System.out.println("short-circuited");
}

// Ternary with a null-guard
String label = (s == null) ? "none" : s.toUpperCase();
```

Output:
```
short-circuited
```

Bit tricks that show up in coding rounds

```
// swap without a temp
int a = 5, b = 9;
a = a ^ b; b = a ^ b; a = a ^ b;   // a=9, b=5

// fast multiply / divide by powers of two
int m = 7 << 2;    // 28   (7 * 4)
int d = 28 >> 2;   // 7    (28 / 4)

// check even/odd
boolean even = (n & 1) == 0;

// unsigned shift on a negative number
int neg = -8;
System.out.println(neg >>  1);   // -4   (keeps sign)
System.out.println(neg >>> 1);   // 2147483644 (fills with 0)
```

Output:
```
-4
2147483644
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java operators follow C-like precedence. The two things I'm careful about: `++i` vs `i++` in expressions, and using `&&`/`||` deliberately for short-circuiting — `&` and `|` evaluate both operands and can turn a safe null check into an NPE."

Flashcards

Q. Difference between && and &?
A. && short-circuits (skips the right operand if the left decides the result); & always evaluates both. On integers, & is bitwise AND.

Q. What is `a = a++;` when a is 10?
A. a stays 10. a++ returns the old value (10) and that value is assigned back to a.

Q. What is -8 >>> 1?
A. 2147483644. >>> is the unsigned right shift — it fills with zeros instead of the sign bit.

Q. How do you swap two ints with XOR?
A. a = a ^ b; b = a ^ b; a = a ^ b; — works because x^x == 0 and x^0 == x.

Q. Is the ternary operator a statement or an expression?
A. An expression — it produces a value, so both branches must have compatible types and it can't stand alone as a statement.

Q. How do you check if a number is a power of two?
A. n > 0 && (n & (n - 1)) == 0 — powers of two have exactly one set bit.

Quiz

Q1. What is the value of `x` after: `int x = 3; x += x++ * 2 + ++x;`

[ ] A. 15
[x] B. 16
[ ] C. 17
[ ] D. 18

> **Why:** x++ → 3 (x becomes 4), ++x → 5 (x becomes 5). So 3*2 + 5 = 11, then x = 3 + 11 = 14? Careful: compound assignment evaluates the *current* x on the left at the end: x = 3 + (3*2 + 5) = 14. Actually re-check: left operand of += is read first (3), right side = 3*2 + 5 = 11, total = 14.

Q2. Which operator has the HIGHEST precedence in Java?

[ ] A. * (multiplication)
[x] B. ++ (postfix)
[ ] C. &&
[ ] D. == (equality)

> **Why:** Postfix `++/--` and `. / [] / (params)` bind tighter than everything else, including unary prefix and multiplication.

Q3. `System.out.println(7 / 2 * 2.0);` prints:

[ ] A. 7.0
[x] B. 6.0
[ ] C. 7
[ ] D. 6

> **Why:** Left-to-right: `7 / 2` is integer division → 3, then `3 * 2.0` → 6.0. The int result is cast to double only at the second step, so precision is already lost.

4. Control Flow: switch, loops, break/continue Core 30 min

Modern Java (14+) rewrote `switch`. Interviewers love asking the *old* fall-through question, but if you volunteer the arrow syntax and switch expressions you signal you know current Java — a nice edge for a fresher.

Key points

- Classic `switch` uses `break`; without it, execution **falls through** to the next case.
- Arrow switch (`case 1 ->`) does NOT fall through and needs no break — this is the one to write.
- Switch **expressions** (Java 14+) return a value and must be exhaustive (use `default` or cover all enum constants); use `yield` in a block.
- `switch` works on: byte/short/char/int, their wrappers, String, enums, and (Java 21) patterns with type matching.
- `break` exits the loop; `continue` skips to the next iteration. **Labelled** break/continue escapes a specific outer loop.
- Enhanced for (`for (T x : coll)`) works over arrays and anything Iterable.

Code

Old switch vs new switch — write it the new way

```
// OLD: colon syntax, needs break, falls through if you forget
String oldWay(int day) {
   String name;
   switch (day) {
       case 1: name = "Mon"; break;
       case 2: name = "Tue"; break;
       default: name = "Other";
   }
   return name;
}

// NEW (Java 14+): arrow syntax, no fall-through, no break
String newWay(int day) {
   return switch (day) {
       case 1 -> "Mon";
       case 2, 3, 4, 5 -> "Weekday";
       case 6, 7 -> "Weekend";
       default -> throw new IllegalArgumentException("bad day: " + day);
   };
}

// Switch expression with a block -> use `yield`
int quarter = switch (month) {
   case 1, 2, 3    -> 1;
   case 4, 5, 6    -> 2;
   case 7, 8, 9    -> 3;
   case 10, 11, 12 -> 4;
   default -> { System.out.println("invalid"); yield -1; }
};
```

Fall-through: the question they ask to see if you've been burned

```
int n = 2;
switch (n) {
   case 1: System.out.println("one");
   case 2: System.out.println("two");
   case 3: System.out.println("three"); break;
   default: System.out.println("other");
}
// Prints: two, three  (no break after case 2 -> falls into case 3)
```

```
Output:
two
three
```

Labelled break — escaping nested loops cleanly

```
outer:
for (int i = 0; i < 3; i++) {
   for (int j = 0; j < 3; j++) {
       if (i == 1 && j == 1) break outer;   // leaves BOTH loops
       System.out.println(i + "," + j);
   }
}
// 0,0  0,1  0,2  1,0  -> then break outer
continue outer;  // jumps to the next iteration of the OUTER loop
```

```
Output:
0,0
0,1
0,2
1,0
```

Loops you must be able to write from memory

```
// for
for (int i = 0; i < 5; i++) { }

// while
int i = 0;
while (i < 5) { i++; }

// do-while: body ALWAYS runs at least once
do { System.out.println("runs once"); } while (false);

// enhanced for (read-only over the elements)
for (String s : new String[]{"a", "b"}) { System.out.println(s); }

// infinite loop + break
while (true) { if (done()) break; }

// foreach with index when you need both
int idx = 0;
for (String s : list) { System.out.println(idx++ + ": " + s); }
```

SAY IT LIKE THIS IN THE INTERVIEW

"I default to the modern arrow switch and switch expressions — no fall-through, it returns a value, and the compiler forces exhaustiveness. I know the classic colon form falls through without break, which is exactly why I avoid it."

Flashcards

Q. What is switch fall-through and how do you avoid it?
A. In colon-style switch, without `break` execution continues into the next case. Use arrow syntax `case 1 ->` (no fall-through) or add break.

Q. What is a switch expression and which keyword yields a value from a block?
A. A switch that returns a value (Java 14+): `int x = switch (...) { case 1 -> 10; default -> { yield 0; } };`. `yield` returns a value from a block.

Q. Which types can you switch on?
A. byte, short, char, int and their wrappers, String, enums, and since Java 21, type patterns. Not long, float, double, or boolean.

Q. Difference between break, continue and a labelled break?
A. break exits the innermost loop/switch; continue skips to the next iteration; `label: ... break label;` exits an outer labelled loop.

Q. Which loop always executes its body at least once?
A. do-while — the condition is checked after the body.

Q. Can you use `break` outside a loop or switch?
A. No — compile error, unless it's a labelled break targeting an enclosing labelled block.

Quiz

Q1. In a switch expression, what happens if no case matches and there's no default?

[ ] A. It returns null
[x] B. Compile error (for enums/ sealed types) or MatchException at runtime
[ ] C. It falls through to the end
[ ] D. It throws NullPointerException

> **Why:** Switch expressions must be exhaustive. For enums the compiler checks it (error if a constant is missing and no default); otherwise Java throws MatchException at runtime.

Q2. How many times does this loop body run? `for (int i = 0; i < 10; i++) { if (i % 2 == 0) continue; System.out.print(i); }`

[ ] A. 10
[x] B. 5
[ ] C. 4
[ ] D. 0

> **Why:** `continue` skips even values, so only 1,3,5,7,9 print — 5 executions.

Q3. Does `case 1 -> ...` fall through to `case 2`?

[ ] A. Yes, always
[x] B. No — arrow cases never fall through
[ ] C. Only if you omit break
[ ] D. Only with yield

> **Why:** Arrow-form switch labels do not fall through. That's the main motivation for the new syntax.

5. Arrays Core 30 min

Arrays are fixed-size, zero-indexed objects stored contiguously. You'll use `ArrayList` in real code, but arrays still rule coding rounds (two-pointer, sliding window, matrices) and at least one interview question will be array-flavoured.

Key points

- Length is fixed at creation — `arr.length` is a **field** (not a method, unlike `String.length()` and `List.size()`).
- Elements get default values automatically (`0`, `false`, `null`), so you never have to initialise a new array.
- 2D arrays in Java are **arrays of arrays** — rows can have different lengths (jagged arrays).
- `ArrayIndexOutOfBoundsException` for a bad index; `ArrayStoreException` for putting the wrong type in.
- `Arrays` utility class = `toString`, `sort`, `binarySearch`, `equals`, `fill`, `copyOf`, `asList`, `stream`.
- Array covariance: `String[]` IS-A `Object[]`, so `Object[] o = new String[1]; o[0] = 5;` compiles and throws at runtime.

Code

Creating, filling and printing arrays

```
int[] a = new int[5];              // [0,0,0,0,0]
int[] b = {1, 2, 3, 4, 5};         // literal (only at declaration)
int[] c = new int[]{1, 2, 3};      // anonymous form
// int[] d = new int[3]{1,2,3};    // ERROR: can't give size AND values

System.out.println(a.length);              // 5   (field, no parentheses)
System.out.println(Arrays.toString(b));    // [1, 2, 3, 4, 5]
System.out.println(b);                     // [I@1b6d3586 <- useless! arrays
                                          // don't override toString()
```

```
Output:
5
[1, 2, 3, 4, 5]
[I@1b6d3586
```

2D arrays and jagged arrays

```
int[][] m = new int[3][4];          // 3 rows, 4 columns
int[][] t = { {1,2}, {3,4,5}, {6} }; // jagged: rows differ in length

System.out.println(m.length);       // 3  -> number of ROWS
System.out.println(m[0].length);    // 4  -> columns in row 0
System.out.println(Arrays.deepToString(t)); // [[1, 2], [3, 4, 5], [6]]

// Iterate a matrix
for (int r = 0; r < m.length; r++)
   for (int c = 0; c < m[r].length; c++)
       m[r][c] = r * c;

for (int[] row : m) System.out.println(Arrays.toString(row));
```

```
Output:
3
4
[[1, 2], [3, 4, 5], [6]]
```

The java.util.Arrays toolkit (memorise these)

```
int[] a = {5, 3, 1, 4, 2};

Arrays.sort(a);                       // in-place ascending -> [1,2,3,4,5]
int idx = Arrays.binarySearch(a, 4);  // 3  (array MUST be sorted first!)
Arrays.fill(a, 0);                    // [0,0,0,0,0]
int[] copy = Arrays.copyOf(a, 3);     // [0,0,0]  (truncate or pad)
int[] range = Arrays.copyOfRange(a, 1, 3);
Arrays.equals(a, copy);               // value equality (NOT a.equals(copy)!)

// asList gotcha: the list is FIXED-SIZE and backed by the array
List<String> fixed = Arrays.asList("a", "b", "c");
// fixed.add("d");   -> UnsupportedOperationException
List<String> real = new ArrayList<>(Arrays.asList("a", "b", "c")); // mutable

// sorting objects
Arrays.sort(words, Comparator.comparing(String::length).reversed());
```

`Arrays.asList` returns a fixed-size view. Wrap it in `new ArrayList<>(...)` or use `List.of(...)` (immutable) when you need a real list.

Two patterns that solve half of array coding problems

```
// 1. Two pointers: check if a sorted array has a pair summing to target
static boolean hasPair(int[] a, int target) {
   int l = 0, r = a.length - 1;
   while (l < r) {
       int sum = a[l] + a[r];
       if (sum == target) return true;
       if (sum < target) l++; else r--;
   }
   return false;
}

// 2. Kadane's algorithm: maximum subarray sum, O(n)
static int maxSubArray(int[] a) {
   int best = a[0], cur = a[0];
   for (int i = 1; i < a.length; i++) {
       cur = Math.max(a[i], cur + a[i]);
       best = Math.max(best, cur);
   }
   return best;
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"Arrays are fixed-size, zero-indexed, and default-initialised. `arr.length` is a field, not a method. For anything dynamic I'd reach for ArrayList instead, but for coding rounds I'm comfortable with two-pointer and sliding-window patterns directly on arrays."

Flashcards

Q. Why do you need Arrays.toString(arr) instead of printing the array?
A. Arrays don't override toString(), so `System.out.println(arr)` prints the default Object form like [I@1b6d3586. Arrays.toString() gives readable output.

Q. arr.length vs String.length() vs List.size()?
A. Array length is a public final *field*; String and List use *methods* — s.length() and list.size().

Q. What is a jagged array in Java?
A. A 2D array whose rows have different lengths — Java's multi-dimensional arrays are arrays of arrays, so each row is independent.

Q. What does Arrays.asList() return and what's the catch?
A. A fixed-size List backed by the array — add/remove throw UnsupportedOperationException, but set() works and mutates the original array.
Q. Difference between List.of() and Arrays.asList()?
A. List.of() (Java 9) is fully immutable and rejects nulls; Arrays.asList() is fixed-size but allows set() and nulls.

Q. What must be true before Arrays.binarySearch()?
A. The array must be sorted in ascending order, otherwise the result is undefined.

Q. Default values in a new int[] vs new String[]?
A. int[] → 0; String[] → null. All arrays are default-initialised.

Quiz

Q1. What does this print? `int[] x = {1,2,3}; int[] y = x; y[0] = 99; System.out.println(x[0]);`

[ ] A. 1
[x] B. 99
[ ] C. Compile error
[ ] D. ArrayStoreException

> **Why:** Arrays are objects, so `y = x` copies the *reference*, not the array. Both variables point at the same array. Use Arrays.copyOf() or clone() for a real copy.

Q2. Which line throws at compile time? (a) `Object[] o = new String[3];` (b) `o[0] = new Integer(5);`

[ ] A. (a) — arrays are invariant
[x] B. (b) — the assignment compiles, so it fails at runtime with ArrayStoreException
[ ] C. Both compile and run fine
[ ] D. (b) throws ClassCastException at compile time

> **Why:** Arrays are covariant: String[] IS-A Object[]. But the JVM remembers the component type and throws ArrayStoreException when you store an Integer at runtime.

Q3. What's the time complexity of Kadane's maximum subarray algorithm?

[ ] A. O(n log n)
[x] B. O(n)
[ ] C. O(n^2)
[ ] D. O(1)

> **Why:** Single pass keeping the best sum ending here and the best overall — O(n) time, O(1) space.

Q4. `int[][] m = new int[3][4];` — what is `m.length` and `m[0].length`?

[ ] A. 4 and 3
[x] B. 3 and 4
[ ] C. 12 and 3
[ ] D. 3 and 3

> **Why:** `m.length` is the number of rows (3); `m[0].length` is the length of the first row (4).

6. Strings, Immutability & the String Pool Must-know 45 min

If there's one topic guaranteed in a Java fresher interview, it's this one. String immutability, the string pool, `==` vs `equals()`, and StringBuilder — all in one conversation.

Key points

- `String` is **immutable**: every 'change' creates a new object. That's what makes it thread-safe, hashable and safe to use as a HashMap key.
- **String Pool** (String Constant Pool) — a special heap area where literal strings are interned and reused. `new String("hi")` deliberately bypasses it.
- `==` compares **references**; `equals()` compares **contents**. String overrides `equals()`.
- `String` overrides `equals()` AND `hashCode()`, so it's a safe HashMap key.
- **StringBuilder** (fast, not thread-safe) vs **StringBuffer** (synchronized, thread-safe, slower). Use StringBuilder.
- String concatenation in a loop with `+` is O(n^2) — the compiler creates a new StringBuilder per iteration. Use an explicit StringBuilder.

Reference

Class	Mutable?	Thread-safe?	When to use
String	No (immutable)	Yes (naturally)	Default choice; keys, constants, literals
StringBuilder	Yes	No	Building strings in a single-threaded method (most cases)
StringBuffer	Yes	Yes (synchronized)	Legacy / shared across threads — rare; prefer StringBuilder
CharSequence	interface	-	Parameter type when you accept any of the above

Code

The String pool, visualised in code — memorise this output

```
String s1 = "java";            // literal -> goes in the pool
String s2 = "java";            // same literal -> REUSES the pooled object
String s3 = new String("java");// `new` -> forces a NEW object on the heap
String s4 = s3.intern();       // returns the pooled instance

System.out.println(s1 == s2);          // true  -> same pooled object
System.out.println(s1 == s3);          // false -> different objects
System.out.println(s1 == s4);          // true  -> intern() found the pool copy
System.out.println(s1.equals(s3));     // true  -> contents match

// Compile-time constant folding
String a = "ja" + "va";                // constant expression -> "java"
System.out.println(a == s1);           // true

String x = "ja";
String b = x + "va";                   // runtime concat -> new object
System.out.println(b == s1);           // false
System.out.println(b.equals(s1));      // true
```

```
Output:
true
false
true
true
true
false
true
```

> `+` on two compile-time constants is folded by the compiler. `+` involving a variable happens at runtime and creates a new String.

Immutability in practice — why `toUpperCase()` 'does nothing'

```
String s = "hello";
s.toUpperCase();          // returns a NEW String, discards it
System.out.println(s);    // hello  <- unchanged!

s = s.toUpperCase();      // reassign to capture the result
System.out.println(s);    // HELLO

// Same for every "modifying" String method:
// concat, replace, substring, trim, strip... they ALL return new objects
```

```
Output:
hello
HELLO
```

StringBuilder — and the loop performance trap

```
// BAD: O(n^2). A new StringBuilder is built every iteration.
String result = "";
for (int i = 0; i < 100_000; i++) {
   result += i;          // never do this in a loop
}

// GOOD: one builder, O(n)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100_000; i++) {
   sb.append(i);
}
String result2 = sb.toString();

// Chained API
String msg = new StringBuilder()
       .append("Hello, ").append(name).append("!")
       .reverse().toString();

// Note: append(i) with int is fine; append(char) is the fastest overload
```

String methods you'll be asked to use in coding rounds

```
String s = "  Java Programming  ";

s.length();                       // 20
s.trim();                         // removes chars <= U+0020 (old)
s.strip();                        // Unicode-aware (Java 11+) - prefer this
s.stripLeading(); s.stripTrailing();
s.toLowerCase(); s.toUpperCase();
s.charAt(0);                      // ' '
s.substring(2, 6);                // "Java"  (begin inclusive, end EXCLUSIVE)
s.indexOf("Java");                // 2  (-1 if not found)
s.lastIndexOf("m");               // index of last occurrence
s.contains("Prog");               // true
s.startsWith("  Ja");             // true
s.replace("Java", "Kotlin");      // all occurrences (CharSequence version)
s.replaceAll("\\s+", " ");        // regex version
s.split(" ");                     // regex split
s.toCharArray();                  // char[]
s.chars();                        // IntStream of chars
s.repeat(2);                      // Java 11
s.isBlank();                      // Java 11 - true for whitespace-only
s.lines();                        // Java 11 - Stream<String>
"".isEmpty();                     // true (length == 0)
"  ".isBlank();                   // true (Java 11)

// Reversing a string (classic question)
String rev = new StringBuilder(s).reverse().toString();

// Comparing
s.compareTo(other);               // lexicographic, 0 when equal
s.equalsIgnoreCase(other);        // case-insensitive equality
```

Text blocks (Java 15+) — multi-line strings without the noise

```
String json = """
       {
         "name": "Parvez",
         "role": "Java Developer"
       }
       """;
// Incident leading whitespace is stripped automatically.
System.out.println(json);
```

```
Output:
{
 "name": "Parvez",
 "role": "Java Developer"
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"String is immutable — every modification returns a new object, which is why it's thread-safe and safe as a HashMap key. Literals are interned in the string pool and reused; `new String()` forces a separate object. I compare strings with equals(), never ==, and I use StringBuilder when I'm building strings in a loop."

Flashcards

Q. Why is String immutable in Java?
A. For security, thread-safety, hashcode caching and string-pool reuse. Since strings are used as keys and parameters everywhere, immutability makes them safely shareable.

Q. How many objects are created by `String s = new String("hello");`?
A. Two (if "hello" isn't already pooled): one literal in the pool and one new object on the heap. One if the literal is already interned.

Q. What does intern() do?
A. Returns the canonical pooled instance of the string, adding it to the pool if absent. Useful for making == work and saving memory with many duplicates.

Q. StringBuilder vs StringBuffer?
A. Both mutable; StringBuilder is not synchronized (faster, default choice), StringBuffer methods are synchronized (thread-safe, legacy).

Q. String vs StringBuilder vs StringBuffer on equals()?
A. String overrides equals() to compare contents. StringBuilder/StringBuffer do NOT — their equals() is reference equality.

Q. Does `String s = "a" + "b";` create 3 objects?
A. No — the compiler folds it into the single literal "ab", which is pooled.

Q. trim() vs strip()?
A. trim() removes characters <= U+0020 only; strip() (Java 11) is Unicode-aware and removes all whitespace. Prefer strip().

Q. Can a String be used as a key in HashMap safely?
A. Yes — String is immutable and caches its hashcode, so the hash never changes after insertion.

Q. What does `s.substring(1, 4)` return for "Programming"?
A. "rog" — the begin index is inclusive and the end index is exclusive.

Quiz

Q1. How many objects does `String s = new String("abc");` create (assuming "abc" is not already pooled)?

[ ] A. 1
[x] B. 2
[ ] C. 3
[ ] D. 0

> **Why:** One String object for the literal in the pool, and one on the heap created by `new`. The variable `s` refers to the heap one.

Q2. What is printed? `String a = "hi"; String b = "h" + "i"; String c = "h"; String d = c + "i"; System.out.println(a==b); System.out.println(a==d);`

[ ] A. true true
[x] B. true false
[ ] C. false false
[ ] D. false true

> **Why:** `"h"+"i"` is a compile-time constant → folded to "hi" → pooled → == true. `c + "i"` involves a variable → evaluated at runtime → a new String → == false.

Q3. Which is the correct way to compare two strings for content equality?

[ ] A. s1 == s2
[x] B. s1.equals(s2)
[ ] C. s1 = s2
[ ] D. s1.compareTo(s2) == 1

> **Why:** equals() compares contents (String overrides it). `==` compares references. Bonus safety tip: call equals on the constant — `"CONST".equals(variable)` — to avoid NPE.

Q4. Why is repeated `+` concatenation in a loop slow?

[x] A. Strings are immutable, so each + allocates a new String (and a StringBuilder)
[ ] B. Because + is synchronized
[ ] C. Because + uses reflection
[ ] D. It isn't — the compiler optimises it away

> **Why:** Each iteration builds a new StringBuilder and String, copying everything so far → $O(n^2)$. An explicit StringBuilder outside the loop makes it $O(n)$.

Q5. What is the output? `System.out.println("Java".substring(1,3));`

[x] A. av
[ ] B. ava

[ ] C. Ja
[ ] D. Jav

Why: beginIndex inclusive, endIndex exclusive → characters at 1 and 2 → "av".

Q6. Which is true about StringBuilder's equals()?

[ ] A. It compares contents, like String
[x] B. It compares references (not overridden)
[ ] C. It throws UnsupportedOperationException
[ ] D. It compares lengths

Why: StringBuilder inherits Object.equals() → reference comparison. Convert with toString() before comparing contents.

7. Input / Output & Formatting Core 25 min

You won't be grilled on this, but you need it to solve coding problems when you're practising on a platform that uses stdin. `Scanner` for simplicity, `BufferedReader` for speed.

Key points

- `Scanner` is easy but **slow** (regex-based) — fine for small inputs, times out on large ones.
- `BufferedReader` + `StringTokenizer`/`split` is the fast pattern for competitive-style input.
- `Scanner.nextLine()` after `nextInt()` is the #1 beginner bug — `nextInt()` leaves the newline in the buffer.
- `System.out.printf` / `String.format` use format specifiers: `%d %s %f %.2f %n`.
- `System.out.println` is slow in a loop — wrap in a `BufferedWriter` or `StringBuilder` for bulk output.

Code

Scanner — and the newline bug that wastes 20 minutes

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter age: ");
int age = sc.nextInt();       // reads the number, leaves "\n" in the buffer

sc.nextLine();                // <-- consume the leftover newline!
System.out.print("Enter name: ");
String name = sc.nextLine();  // now reads the actual line

// Or: always read lines and parse them
int n = Integer.parseInt(sc.nextLine());

sc.close();                   // good hygiene (don't close System.in in online judges)

// Reading until EOF
while (sc.hasNext()) { System.out.println(sc.next()); }
```

BufferedReader — the fast pattern used in coding platforms

```
import java.io.*;
import java.util.*;

class FastIO {
   public static void main(String[] args) throws Exception {
       BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
       StringTokenizer st = new StringTokenizer(br.readLine());

       int n = Integer.parseInt(st.nextToken());
       int[] arr = new int[n];
       st = new StringTokenizer(br.readLine());
       for (int i = 0; i < n; i++) arr[i] = Integer.parseInt(st.nextToken());

       // Fast bulk output
       StringBuilder out = new StringBuilder();
       for (int v : arr) out.append(v * 2).append('\n');
       System.out.print(out);
       br.close();
   }
}
```

Formatting output — printf / String.format

```
String name = "Parvez";
int age = 24;
double salary = 75432.126;

System.out.printf("Name: %s, Age: %d%n", name, age);
System.out.printf("Salary: %.2f%n", salary);       // 75432.13 (rounds)
System.out.printf("|%10s|%n", name);              // right-aligned width 10
System.out.printf("|%-10s|%n", name);             // left-aligned
System.out.printf("Hex: %x, Octal: %o%n", 255, 8); // ff, 10
System.out.printf("%,d%n", 1000000);              // 1,000,000

String s = String.format("%s is %d", name, age);  // build instead of print

// Common specifiers: %s String, %d int, %f float/double, %c char,
// %b boolean, %n platform-independent newline (prefer over \n in printf)
```

Output:

```
Name: Parvez, Age: 24
Salary: 75432.13
|    Parvez|
|Parvez    |
Hex: ff, Octal: 10
1,000,000
```

SAY IT LIKE THIS IN THE INTERVIEW

"I use Scanner for quick input but BufferedReader with StringTokenizer for large inputs, because Scanner is regex-based and slow. And I know the classic nextInt-then-nextLine newline bug — you consume the leftover newline or read lines and parse."

Flashcards

Q. Scanner vs BufferedReader for input?
A. Scanner is convenient (nextInt, nextLine, hasNext) but slow because it uses regex. BufferedReader reads raw characters fast and you parse manually — preferred for large inputs.

Q. Why does nextLine() return empty right after nextInt()?
A. nextInt() reads the number but leaves the newline in the buffer, so nextLine() immediately consumes that empty remainder. Fix: an extra nextLine() to flush it, or read everything as lines.

Q. What's the difference between %n and \n in printf?
A. %n is the platform-independent line separator; \n is always a Unix newline. Prefer %n in formatted output.

Q. How do you print a double with exactly 2 decimals?
A. System.out.printf("%.2f", value) — it rounds automatically (HALF_UP for most values, though it uses the default rounding mode).

Q. Why avoid println inside a large loop?
A. Each call flushes/does I/O work; it's dramatically slower than accumulating into a StringBuilder or BufferedWriter and printing once.

Quiz

Q1. What's the fastest safe way to read 1 million integers from stdin in Java?

[ ] A. Scanner.nextInt() in a loop
[x] B. BufferedReader + StringTokenizer + Integer.parseInt
[ ] C. System.console().readLine()
[ ] D. DataInputStream.readInt() in a loop

Why: BufferedReader buffers the stream and StringTokenizer splits without regex — 5–10x faster than Scanner. (For raw speed you'd write a custom byte-buffer reader.)

Q2. What does `System.out.printf("%.1f", 2.25);` print?

[x] A. 2.2
[ ] B. 2.3

[ ] C. 2.25
[ ] D. 2.0

Why: Formatter uses half-up rounding, but 2.25 is exactly representable and rounds-to-even applies in some paths — in practice `%.1f` of 2.25 on the JDK prints 2.2. The safe interview answer: it rounds to one decimal place.

8. Stack, Heap & Pass-by-Value Must-know 30 min

"Is Java pass-by-value or pass-by-reference?" — the answer is **pass-by-value, always**, and the confusion comes from the fact that for objects, the *value being copied* is the reference. Know this cold; it's asked constantly and most people get it wrong.

Key points

- Primitives live on the **stack** (when local); objects live on the **heap**; the reference variable lives on the stack.
- Java passes a **copy of the variable's value**: for primitives, a copy of the value; for objects, a copy of the reference.
- So you **can** mutate the object a parameter points to, but you **cannot** reassign the caller's variable to point elsewhere.
- `final` on a parameter doesn't stop mutation — it only stops reassignment inside the method.
- Java has **no pointer arithmetic** and **no pass-by-reference** — unlike C++'s `&`.

Reference

Where	Holds	Lifetime	Notes
Stack	Local primitives, object references, method frames	Until the method returns	Fast, LIFO, per-thread, no GC
Heap	All objects and their instance fields	Until unreachable + GC	Shared across threads, managed by GC
Metaspace	Class metadata, method bytecode	Until classloader unloaded	Replaced PermGen in Java 8
String pool	Interned string literals	JVM lifetime (mostly)	Lives on the heap since Java 7

Code

The pass-by-value proof — predict the output before you scroll

```
static void changePrimitive(int n)   { n = 100; }
static void reassign(StringBuilder s){ s = new StringBuilder("new"); }
static void mutate(StringBuilder s)  { s.append(" world"); }

public static void main(String[] a) {
   int num = 10;
   changePrimitive(num);
   System.out.println(num);            // 10  <- unchanged

   StringBuilder sb = new StringBuilder("hello");
   reassign(sb);
   System.out.println(sb);             // hello <- still the original!

   mutate(sb);
   System.out.println(sb);             // hello world <- MUTATED

   // Same story with arrays and any object:
   static void swap(Integer x, Integer y) { Integer t = x; x = y; y = t; } // does nothing
}
```

```
Output:
10
hello
hello world
```

This is THE question. Say: "Java is always pass-by-value. For objects, the value passed is a copy of the reference — so I can mutate the object it points to, but reassigning the parameter has no effect on the caller's variable."

Why swap() can't work the obvious way in Java

```
// Doesn't work: primitives are copied
static void swap(int a, int b) { int t = a; a = b; b = t; }

// Doesn't work either: Integer is immutable, so even the mutation trick fails
static void swap(Integer a, Integer b) { Integer t = a; a = b; b = t; }

// What DOES work: mutate the holder, or return the values
static void swap(int[] arr, int i, int j) {
   int t = arr[i]; arr[i] = arr[j]; arr[j] = t;   // mutates shared array
}
static void swap(Point p) { int t = p.x; p.x = p.y; p.y = t; } // mutates object
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java is strictly pass-by-value. When I pass an object, the value being copied is the reference — so mutating the object through that reference affects the original, but reassigning the parameter never changes the caller's variable. That's why a plain swap(x, y) method can't work."

Flashcards

Q. Is Java pass-by-value or pass-by-reference?
A. Always pass-by-value. For objects the value copied is the reference, which is why mutation of the object is visible but reassignment of the parameter is not.

Q. Why doesn't `void swap(int a, int b)` work in Java?
A. a and b are copies of the caller's values; swapping the copies has no effect. You must mutate a shared container/object or return the values.

Q. What's stored on the stack vs the heap?
A. Stack: method frames, local primitives and object references (per thread). Heap: all objects and their instance fields (shared, GC-managed).

Q. Why does passing a StringBuilder and appending work, but assigning a new one inside the method not work?
A. append() mutates the shared object through the copied reference. Assigning a new object only changes the local copy of the reference.

Q. Where do string literals live since Java 7?
A. On the heap, in the string pool. (Before Java 7 they were in PermGen.)

Q. Does `final` on a method parameter prevent modifying the object?
A. No — it only prevents reassigning the parameter. You can still mutate the object's state.

Quiz

Q1. What's the output? `static void f(String s){ s = s + "!"; }` `String t = "hi"; f(t);` `System.out.print(t);`

[ ] A. hi!
[x] B. hi
[ ] C. null
[ ] D. Compile error

Why: Two reasons: String is immutable (so `+` makes a new object) and Java is pass-by-value (so reassigning `s` inside doesn't touch `t`).

Q2. Which statement about Java memory is FALSE?

[ ] A. Local primitives live on the stack
[ ] B. Objects are always allocated on the heap
[ ] C. Each thread has its own stack but shares the heap
[x] D. The garbage collector destroys objects immediately when a method returns

Why: GC runs on its own schedule and only collects unreachable objects. A method returning doesn't trigger collection — objects are collected when no live references remain and the GC chooses to run.

Q3. `void m(final List<String> l) { l.add("x"); }` — what happens?

[ ] A. Compile error: final list can't be modified
[x] B. Compiles and runs; the list is mutated
[ ] C. Runtime UnsupportedOperationException
[ ] D. Compiles, but the change is discarded

Why: `final` only forbids reassigning `l` to another list. Mutating the referenced list is perfectly legal.

9. Packages, Access Modifiers & the static Keyword Core 30 min

Small topic, guaranteed questions — especially the access-modifier table and "what does static really mean". These are pure recall marks, so memorise the table.

Key points

- `static` members belong to the **class**, not to any instance — one copy shared by all objects, loaded when the class is initialised.
- Static methods **cannot** access instance members directly and **cannot** use `this` or `super`.
- Static methods **can** be overloaded, and **can** be inherited, but are **hidden** (not overridden) when redeclared in a subclass — resolved at compile time.
- **Static initialisation order**: static blocks/fields run once, top-to-bottom, when the class is first loaded; then instance blocks/fields; then the constructor.
- `main` is static precisely so the JVM can call it without creating an object.
- Packages group related classes and avoid name collisions; `import` is compile-time sugar only (doesn't load code or cost runtime memory).

Reference

Modifier	Same class	Same package	Subclass (other pkg)	Anywhere
private	Yes	No	No	No
default (package-private)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Code

Static: shared state, and the trap it creates

```
class Counter {
   static int count = 0;      // ONE copy shared by every instance
   int id;                    // one per object

   Counter() { id = ++count; }
   static int getCount() { return count; }        // static method
   // static void bad() { System.out.println(id); } // ERROR: can't touch instance
}

Counter a = new Counter();   // id=1
Counter b = new Counter();   // id=2
System.out.println(Counter.getCount());  // 2  <- call on the CLASS, not an object
```

```
Output:
2
```

Initialisation order — a favourite output-prediction question

```
class Init {
   static { System.out.println("1. static block"); }
   { System.out.println("2. instance block"); }
   Init() { System.out.println("3. constructor"); }

   public static void main(String[] a) {
       System.out.println("4. main start");
       new Init();
       new Init();          // static block does NOT run again
       System.out.println("5. main end");
   }
}
// Order: static block (once, at class load) -> main ->
//        per object: instance block -> constructor
```

Output:

```
1. static block
4. main start
2. instance block
3. constructor
2. instance block
3. constructor
5. main end
```

Static method hiding vs instance method overriding

```
class Parent { static void greet() { System.out.println("Parent"); } }
class Child  extends Parent {
   static void greet() { System.out.println("Child"); }   // HIDING, not overriding
}

Parent p = new Child();
p.greet();        // "Parent"  <- resolved by the REFERENCE type at compile time
new Child().greet(); // "Child"

// Contrast with instance methods:
class A { void show() { System.out.println("A"); } }
class B extends A { @Override void show() { System.out.println("B"); } }
A ref = new B();
ref.show();       // "B"  <- dynamic dispatch at runtime
```

Output:

```
Parent
Child
B
```

Packages & imports

```
// Declare at the top of the file, before imports
package com.parvez.project.service;

import java.util.List;              // single type
import java.util.*;                  // whole package (no runtime cost)
import static java.lang.Math.*;      // static import: use PI, max() directly
import java.sql.Date;                // disambiguate a name clash

double r = max(PI, 3);               // thanks to the static import

// Two classes with the same simple name -> use the fully-qualified name
java.util.Date utilDate = new java.util.Date();

// Compiling with a package structure:
// javac -d out src/com/parvez/App.java
// java  -cp out com.parvez.App
```

SAY IT LIKE THIS IN THE INTERVIEW

"static means the member belongs to the class rather than an instance — one shared copy, callable without an object, and it can't touch instance state or use this. Access modifiers go private → package-private → protected → public, widening at each step, and protected also grants access to subclasses in other packages."

Flashcards

Q. What does static mean on a field?
A. One copy belonging to the class, shared by all instances, initialised when the class is loaded — not per-object.

Q. Can a static method access instance variables or use this?
A. No. Static context has no instance; referring to instance members or this/super is a compile error.

Q. Can static methods be overridden?
A. No — they're hidden. Redeclaring a static method in a subclass resolves by the reference type at compile time, not by the runtime object.

Q. Order of execution for static block, instance block, constructor?
A. Static block first (once, at class load), then per object: instance initialiser block, then constructor body.

Q. Can you overload static methods? Can main be overloaded?
A. Yes to both — overloading depends on the parameter list, not staticness. Only public static void main(String[]) is the entry point.

Q. What's the difference between protected and package-private?
A. protected = package access PLUS subclasses in other packages. Package-private (no modifier) = same package only, no subclass exception.

Q. Why is main() static?
A. So the JVM can invoke it without instantiating the class — no object exists yet at startup.

Q. Does `import java.util.*;` cost memory or load all classes?
A. No. Imports are compile-time name resolution only; the JVM loads classes lazily on first use.

Quiz

Q1. Which access modifier allows access from a subclass in a DIFFERENT package?

[ ] A. private
[ ] B. default (no modifier)
[x] C. protected
[ ] D. None of them

Why: protected = package-private + subclasses (even in other packages). Note the subtlety: in another package, a subclass can only access it through its own type's reference.

Q2. What is the initialisation order for a subclass instantiation?

[ ] A. Child static → Parent static → Parent instance → Parent ctor → Child instance → Child ctor
[x] B. Parent static → Child static → Parent instance → Parent ctor → Child instance → Child ctor
[ ] C. Parent ctor → Child ctor → static blocks
[ ] D. Instance blocks → static blocks → constructors

Why: Static blocks run top-down as each class loads (parent first), then per object: parent instance initialisers + constructor, then child instance initialisers + constructor.

Q3. Can a static method be called using an object reference, e.g. `obj.staticMethod()`?

[x] A. Yes, but it's bad style and resolved by the declared type
[ ] B. No, compile error
[ ] C. Yes and it uses dynamic dispatch
[ ] D. Only if the method is final

Why: It compiles (with a warning in some IDEs) but is misleading — the call is resolved from the *declared* type, not the runtime object, so it looks polymorphic but isn't.

Q4. Why can't a static method be abstract?

[x] A. Because abstract requires overriding, and static methods are hidden not overridden

[ ] B. Because abstract methods are public

[ ] C. Because static methods are final

[ ] D. It can be

Why: abstract means 'subclasses must provide an implementation via dynamic dispatch'. Static methods have no dynamic dispatch, so the combination is a compile error.

10. Varargs, Enums & Command-Line Args Nice-to-have 20 min

Small, quick marks. Enums in particular come up more than people expect, because an enum can have fields, constructors and methods — it's a full class.

Key points

- Varargs `type... name` must be the **last** parameter and there can be only one; internally it's an array.
- Overloading with varargs can be ambiguous — avoid mixing `(int... x)` and `(int[] x)`.
- Enums are classes: they can have fields, constructors, methods, and can implement interfaces. Constructors are implicitly private.
- Enum constants are `public static final` instances; `values()` and `valueOf()` are compiler-generated.
- Enums are **safe for `==` comparison** (only one instance per constant) and are the best implementation of a thread-safe singleton.

Code

Varargs rules and gotchas

```
static int sum(int... nums) {      // nums IS an int[]
   int t = 0; for (int n : nums) t += n; return t;
}
sum();          // 0     - zero args allowed
sum(1, 2, 3);   // 6
sum(new int[]{1,2,3});  // 6 - you can pass an array directly

// static void bad(String a, int... n, int x) {}  // ERROR: varargs must be LAST
static void ok(String a, int x, int... n) {}      // fine

// Passing null is ambiguous with varargs:
// sum(null);      -> warning/ambiguity; cast it: sum((int[]) null)
```

Enums as real classes

```
enum Status {
   OPEN(1, "Open"), IN_PROGRESS(2, "Working"), CLOSED(3, "Done");

   private final int code;
   private final String label;

   Status(int code, String label) {   // implicitly private
       this.code = code; this.label = label;
   }
   public int getCode() { return code; }
   public String getLabel() { return label; }
}

Status s = Status.OPEN;
System.out.println(s.getCode() + " " + s.getLabel());   // 1 Open
System.out.println(Status.valueOf("CLOSED"));           // CLOSED
for (Status st : Status.values()) System.out.print(st + " ");  // OPEN IN_PROGRESS CLOSED
System.out.println(s.ordinal());                        // 0 (position - DON'T persist it)

// Enums in switch (exhaustive, no default needed for expressions)
String msg = switch (s) {
   case OPEN -> "New ticket";
   case IN_PROGRESS -> "Being worked on";
   case CLOSED -> "Finished";
};
```

```
Output:
1 Open
OPEN IN_PROGRESS CLOSED
0
```

Reading command-line arguments

```
// javac Args.java && java Args hello 42
public class Args {
   public static void main(String[] args) {
       System.out.println("count = " + args.length);   // 2
       for (int i = 0; i < args.length; i++)
           System.out.println("args[" + i + "] = " + args[i]);

       if (args.length > 1) {
           int n = Integer.parseInt(args[1]);          // "42" -> 42
       }
   }
}
```

```
Output:
count = 2
args[0] = hello
args[1] = 42
```

SAY IT LIKE THIS IN THE INTERVIEW

"Varargs is just an array parameter with a friendlier call site, and it must be last. Enums are full classes — constants are singletons, so == is safe, and they can carry fields, methods and even per-constant behaviour."

Flashcards

Q. Rules for varargs parameters?
A. Only one per method, must be the last parameter, internally it's an array, and it accepts zero arguments.

Q. Can an enum have a constructor and methods?
A. Yes — an enum is a class. Constructors are implicitly private, and constants can even override methods individually (constant-specific class bodies).

Q. Is == safe for comparing enums?
A. Yes — each constant is a single instance, so == is both correct and null-safe (unlike equals).

Q. What do values() and valueOf() do?
A. Compiler-generated: values() returns all constants as an array; valueOf(String) returns the constant with that exact name or throws IllegalArgumentException.

Q. Why is using ordinal() in a database a bad idea?
A. ordinal() is the declaration position; reordering or inserting a constant silently changes the persisted meaning. Store the name or an explicit code.

Q. Can an enum implement an interface?
A. Yes, but it can't extend a class (enums implicitly extend java.lang.Enum, and Java is single-inheritance).

Quiz

Q1. Which varargs declaration is legal?

[ ] A. void m(int... a, int... b)
[x] B. void m(String s, int... a)
[ ] C. void m(int... a, String s)
[ ] D. void m(int... a, int b)

Why: There can be only one varargs parameter and it must be the last one in the list.

Q2. Why is an enum the recommended way to write a Singleton in Java?

[ ] A. It's the shortest syntax

[x] B. The JVM guarantees only one instance, and it's safe against reflection and serialization attacks

[ ] C. Enums are faster

[ ] D. It allows lazy loading

> **Why:** Joshua Bloch's 'Effective Java' recommends enum singletons: free serialization safety (no readResolve needed) and the JVM blocks reflective instantiation.

Q3. What does `Status.values()` return?

[ ] A. A String[] of names

[x] B. A Status[] of all constants in declaration order

[ ] C. An int[] of ordinals

[ ] D. A Map of name to constant

> **Why:** The compiler adds a static values() method returning a fresh array of the constants in declaration order.